



UNIVERSITY OF BUEA

Faculty Of Engineering and Technology

CEF440

Internet Programming and Mobile Programming

Group 16

TASK 6: DATABASE DESIGN AND IMPLEMENTATION

Submitted to:

Dr. Nkemeni Valery

2025/2026

GROUP MEMBERS

NKENG SAMA MOKOM	FE23A118
ENOW JOHN EYONG	FE23A046
CHIANGOIESEH CHRIS RAWLINGS	FE23A029
KEUKANG CARMEN ALICIA	FE23A074
NKENGAFAC WILLYHERMAN FONKEM	FE23A119

Table of Contents

1.	INTRODUCTION.....	1
2.	DATA ELEMENTS	1
2.1.	Network Metric Data	1
2.2.	User Feedback Data.....	2
3.	CONCEPTUAL DESIGN.....	3
3.1.	Entities	3
3.2.	Entity Attributes	4
3.3.	Relationships	4
4.	ENTITY RELATIONSHIP (ER) DIAGRAM.....	5
5.	DATABASE IMPLEMENTATION	7
5.1.	MongoDB Atlas Configuration	7
5.2.	Mongoose Schema – NetworkMetric	9
5.3.	Mongoose Schema – Feedback	9
6.	BACKEND IMPLEMENTATION.....	10
6.1.	Technology Stack	10
6.2.	Entry Point – server.js	10
6.3.	API Endpoints – Network Metrics	11
6.4.	API Endpoints – Feedback	12
6.5.	Input Validation.....	12
7.	CONNECTING BACKEND WITH DATABASE.....	13
7.1.	DB.js – Connection Module	13
7.2.	Environment Variable Configuration	13
7.3.	Connection Lifecycle.....	15
	Security Considerations	16
8.	CONNECTING BACKEND TO FRONT END	17
9.	CONCLUSION	19
10.	REFERENCES	20

1. INTRODUCTION

NetInsight is a mobile application built with React Native (Expo) designed to anonymously collect and analyse mobile network quality of experience (QoE) data across the city of Buea, Cameroon. The application measures download speed, upload speed, latency, and geographic location in real time, then transmits these measurements to a cloud-hosted backend for persistent storage and aggregation.

This report describes the complete database design and backend implementation underpinning NetInsight. It covers the identification of data elements, conceptual and logical data modelling, the Entity-Relationship (ER) diagram, the physical database implementation using MongoDB Atlas, the Express.js backend API implementation, and the mechanism by which the database is connected to the backend service deployed on Railway.

System Overview

- Frontend: React Native (Expo)
- Backend: Node.js + Express.js 5
- Database: MongoDB Atlas (cloud)
- Hosting: Railway
- ODM: Mongoose 8

2. DATA ELEMENTS

The NetInsight system collects and persists two distinct categories of data, each serving a specific purpose within the application's overall objectives.

2.1. Network Metric Data

Network metric data is collected automatically and continuously by the mobile application at periodic intervals (approximately every five minutes) or on demand when the user opens the dashboard. Each measurement captures the following data elements:

Field Name	Data Type	Unit / Format	Description
deviceId	String	Text	Unique identifier for the reporting device (e.g. "device-ios")

timestamp	Date	ISO 8601	Exact moment the measurement was taken
connectionType	String	Enum	Network type: wifi, cellular, ethernet, unknown
isConnected	Boolean	true / false	Whether the device has an active network connection
isInternetReachable	Boolean	true / false	Whether the internet is reachable beyond the local network
signalStrength	Number	dBm	Radio signal strength in decibels-milliwatts (device-dependent)
downloadSpeed	Number	Mbps	Measured download throughput in megabits per second
downloadStatus	String	Enum	Qualitative classification: poor / fair / good / excellent
uploadSpeed	Number	Mbps	Measured upload throughput in megabits per second
uploadStatus	String	Enum	Qualitative classification: poor / fair / good / excellent
latency	Number	Milliseconds	Round-trip latency measured via HTTP ping to google.com/generate_204
latencyStatus	String	Enum	Qualitative classification: poor / fair / good / excellent
location.latitude	Number	Decimal degrees	WGS-84 geographic latitude
location.longitude	Number	Decimal degrees	WGS-84 geographic longitude
location.accuracy	Number	Metres	GPS accuracy radius reported by the device

2.2.User Feedback Data

User feedback data is submitted voluntarily by the application's users through the Feedback

screen. It captures subjective quality assessments linked to geographic locations within Buea. The following data elements are recorded:

Field Name	Data Type	Unit / Format	Description
experience	String	Enum	Overall experience: very_poor / poor / fair / good / excellent
areaOfFeedback	String	Text	Category of feedback: data_speed, signal_strength, call_quality, etc.
description	String	Text	Free-text description of the user's experience (optional)
rating	Number	1 – 5	Numeric star rating from 1 (worst) to 5 (best)
location.latitude	Number	Decimal degrees	Geographic latitude of the reported area
location.longitude	Number	Decimal degrees	Geographic longitude of the reported area
networkProvider	String	Text	Mobile network provider (defaults to "Anonymous")
resolved	Boolean	true / false	Admin flag indicating whether the feedback has been actioned
createdAt	Date	ISO 8601	Automatic timestamp added by Mongoose on document creation
updatedAt	Date	ISO 8601	Automatic timestamp updated by Mongoose on each modification

3. CONCEPTUAL DESIGN

The conceptual design phase identifies the high-level entities within the NetInsight system, their attributes, and the relationships between them, independent of any specific database technology or implementation detail.

3.1. Entities

This system is centred around two primary entities:

- **NetworkMetric:** Represents a single automated network quality measurement captured by a device at a specific point in time and geographic location. It is the core data entity and grows continuously as the application runs on users' devices.
- **Feedback:** Represents a voluntary, subjective quality assessment submitted by a user. It links a user's perception of network quality to a geographic area within Buea and a specific category of concern.

3.2. Entity Attributes

NetworkMetric Attributes:

- deviceId (identifier)
- timestamp, connectionType, isConnected, isInternetReachable
- signalStrength, downloadSpeed, downloadStatus
- uploadSpeed, uploadStatus, latency, latencyStatus
- location (latitude, longitude, accuracy) – embedded sub-object
- createdAt, updatedAt (system-managed)

Feedback Attributes:

- experience, areaOfFeedback, description, rating
- location (latitude, longitude) – embedded sub-object
- networkProvider, resolved
- createdAt, updatedAt (system-managed)

3.3. Relationships

The two entities are logically related but are stored independently:

The cardinalities are:

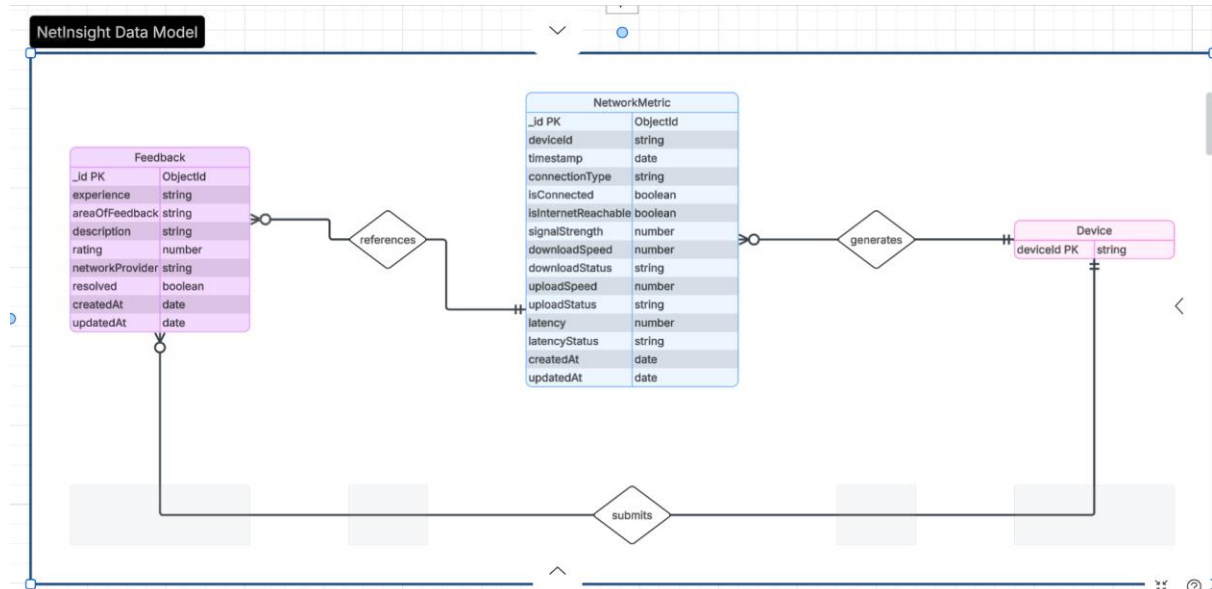
- Device (1) —< NetworkMetric (many): one device produces many metric samples
- User (1) —< Feedback (many): one user can submit many feedback entries
- NetworkMetric (1) — Location (1): each metric embeds exactly one location
- Feedback (1) — Location (1): each feedback entry embeds exactly one location

Relationship Summary

A Device (identified by deviceId) generates zero or more NetworkMetric records over time.

A User submits zero or more Feedback records. Both NetworkMetric and Feedback records

embed a Location sub-object rather than referencing a separate Location entity, which is appropriate for a document-oriented database and avoids unnecessary joins. There is no enforced foreign-key relationship between the two collections in MongoDB; they are queried independently.



4. ENTITY RELATIONSHIP (ER) DIAGRAM

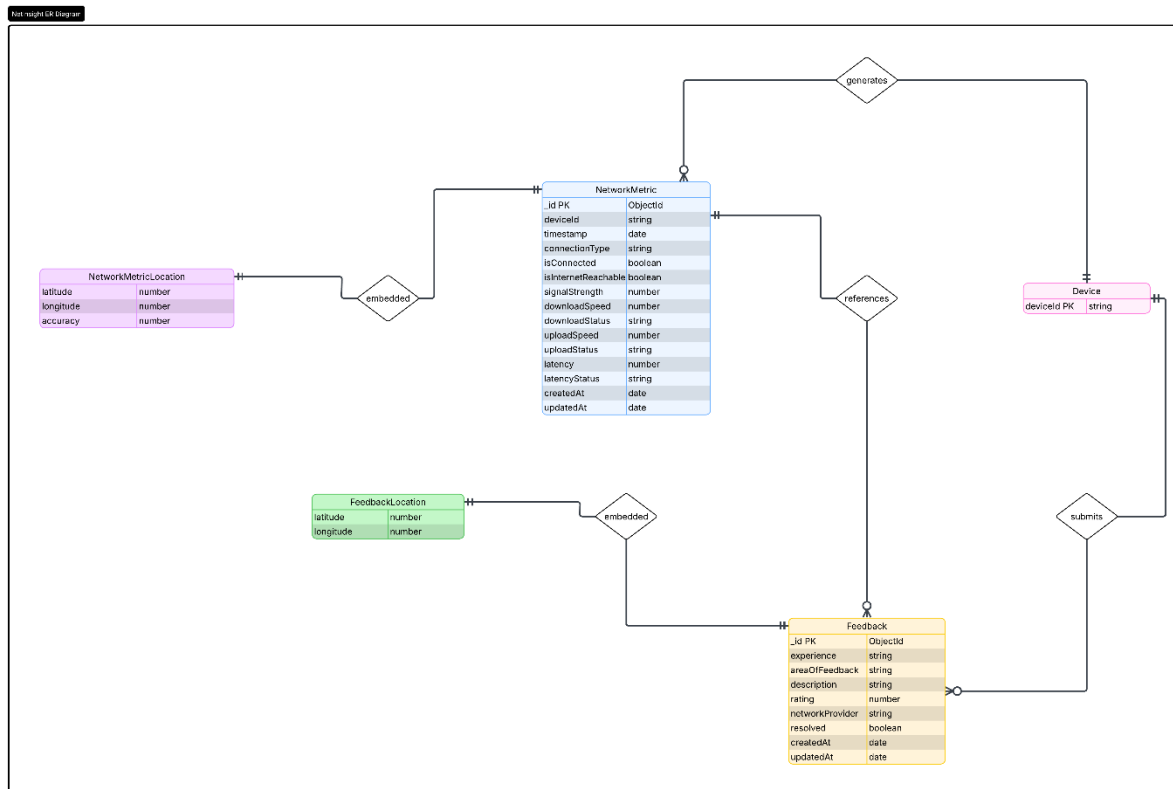
The following Entity-Relationship diagram depicts the two main entities in the NetInsight database, their attributes, and the relationships between them. Because MongoDB is a document-oriented NoSQL database, embedded sub-documents (Location) are shown as dependent entities rather than separate tables.

NetInsight Entity Relationship Diagram

ENTITY: NetworkMetric (Collection: networkmetrics)		
Attribute	Type	Constraints
<code>_id</code>	ObjectId	PRIMARY KEY, auto-generated
<code>deviceId</code>	String	REQUIRED
<code>timestamp</code>	Date	REQUIRED, default: now
<code>connectionType</code>	String	default: "unknown"
<code>isConnected</code>	Boolean	default: false
<code>isInternetReachable</code>	Boolean	default: false

signalStrength	Number (dBm)	default: -100
downloadSpeed	Number (Mbps)	default: 0
downloadStatus	String	poor fair good excellent
uploadSpeed	Number (Mbps)	default: 0
uploadStatus	String	poor fair good excellent
latency	Number (ms)	default: 0
latencyStatus	String	poor fair good excellent
location.latitude	Number	Embedded, default: 0
location.longitude	Number	Embedded, default: 0
location.accuracy	Number	Embedded, default: 0
createdAt	Date	Auto-managed by Mongoose
updatedAt	Date	Auto-managed by Mongoose

ENTITY: Feedback (Collection: feedbacks)		
Attribute	Type	Constraints
_id	ObjectId	PRIMARY KEY, auto-generated
experience	String	REQUIRED, ENUM: very_poor...excellent
areaOfFeedback	String	REQUIRED
description	String	optional, default: ""
rating	Number	REQUIRED, min:1 max:5
location.latitude	Number	Embedded, default: 0
location.longitude	Number	Embedded, default: 0
networkProvider	String	default: "Anonymous"
resolved	Boolean	default: false
createdAt	Date	Auto-managed by Mongoose
updatedAt	Date	Auto-managed by Mongoose



5. DATABASE IMPLEMENTATION

The NetInsight database is implemented using MongoDB Atlas, a fully managed cloud database service. MongoDB was selected over a relational database for the following reasons:

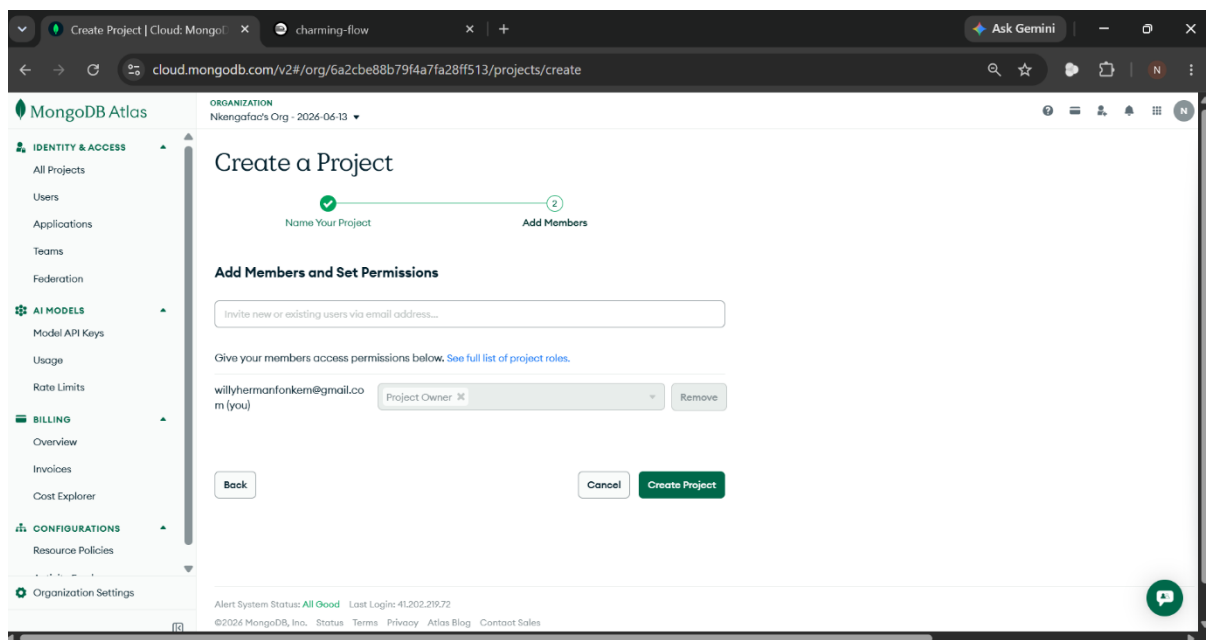
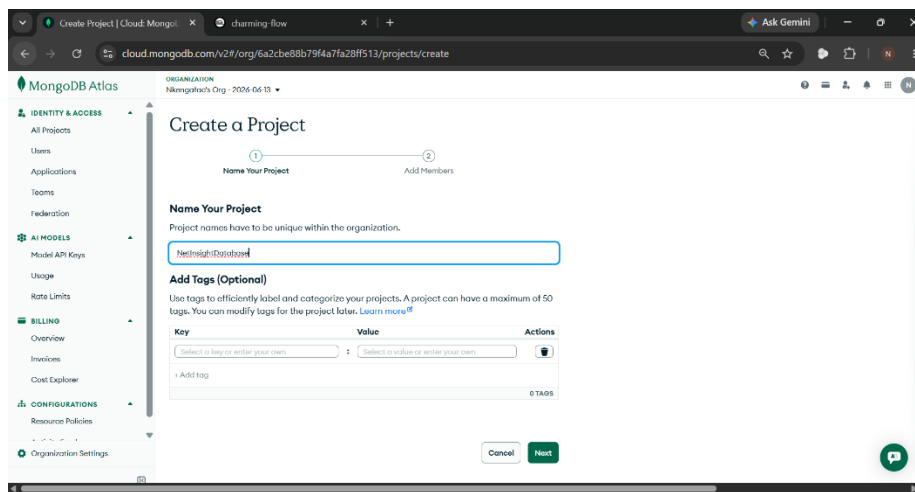
- **Schema flexibility:** Network metric payloads may evolve over time (e.g. new fields added as sensors improve). MongoDB's document model accommodates schema changes without expensive migrations.
- **Embedded documents:** The Location sub-object is a natural fit for MongoDB's nested document model, avoiding the need for JOIN operations.
- **Geospatial capability:** MongoDB has native support for geospatial indexes and queries, which can be leveraged in future versions to query metrics by area.
- **Scalability:** MongoDB Atlas scales horizontally and handles high write throughput, appropriate for a system where many devices continuously write measurement samples.

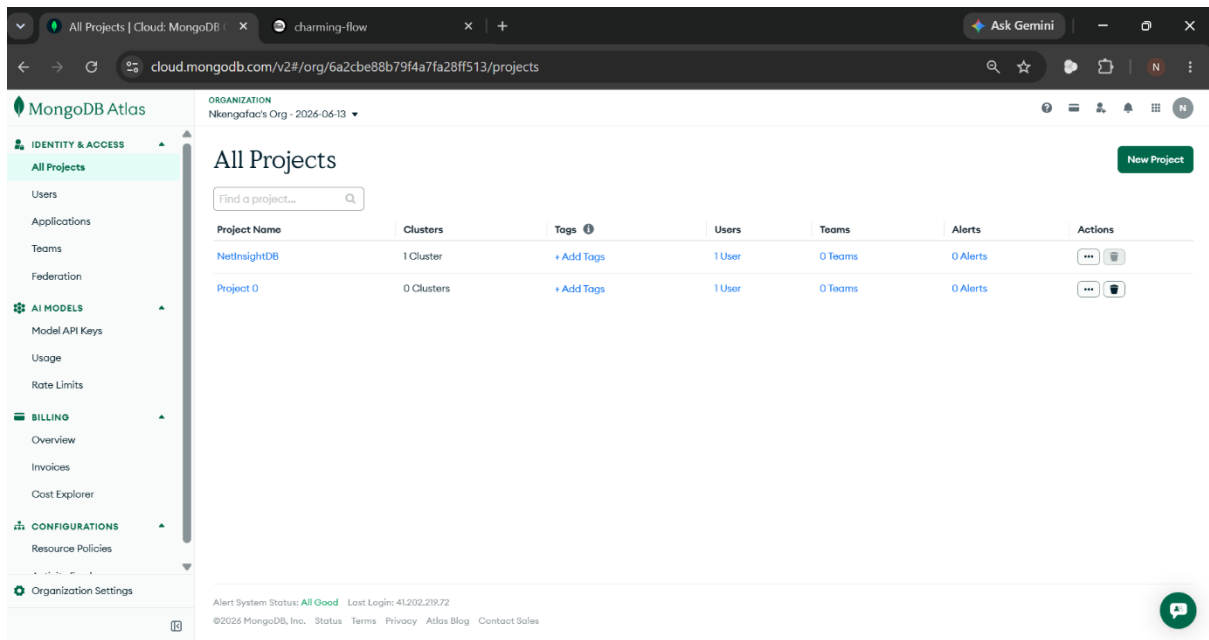
5.1. MongoDB Atlas Configuration

The production database is hosted on a free-tier M0 cluster on MongoDB Atlas (cluster name:

Cluster0, project: NetInsightDB). The following configuration decisions were made:

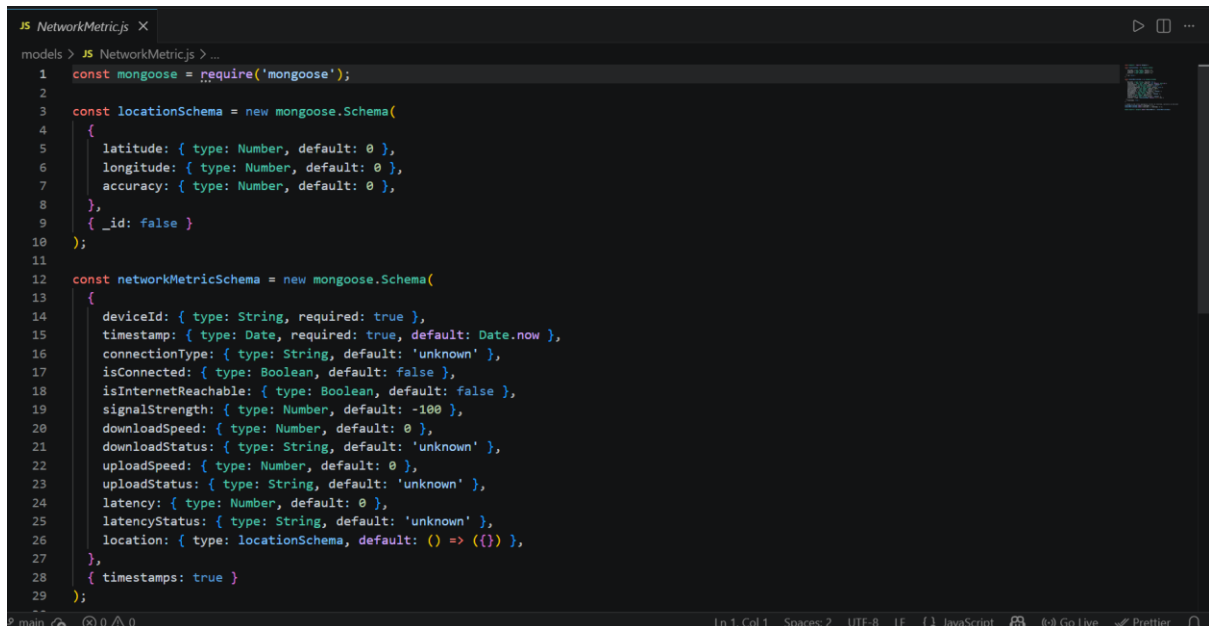
- **Database name:** NetInsight.
- **Collections:** networkmetrics and feedbacks (Mongoose pluralises and lowercases the model names automatically).
- **Network Access:** 0.0.0.0/0 allowed (required because Railway's outbound IPs are not static).
- **Authentication:** SCRAM-SHA-256 username/password, credentials stored exclusively as a Railway environment variable (MONGODB_URI).





5.2. Mongoose Schema – NetworkMetric

The NetworkMetric schema is defined in models/NetworkMetric.js using Mongoose's Schema API. An embedded locationSchema sub-document is used for the geographic coordinates:



5.3. Mongoose Schema – Feedback

The Feedback schema is defined in models/Feedback.js. It uses an enum constraint on the experience field to enforce data integrity at the application layer, and a min/max constraint on rating:

```

1  const mongoose = require('mongoose');
2
3  const feedbackLocationSchema = new mongoose.Schema({
4    latitude: { type: Number, default: 0 },
5    longitude: { type: Number, default: 0 },
6  },
7  { _id: false }
8  );
9
10 const feedbackSchema = new mongoose.Schema({
11   experience: {
12     type: String,
13     enum: ['very_poor', 'poor', 'fair', 'good', 'excellent'],
14     required: true,
15   },
16   areaOfFeedback: { type: String, required: true },
17   description: { type: String, default: '' },
18   rating: { type: Number, min: 1, max: 5, required: true },
19   location: { type: feedbackLocationSchema, default: () => ({}) },
20   networkProvider: { type: String, default: 'Anonymous' },
21   resolved: { type: Boolean, default: false },
22   timestamps: true
23 });
24
25 feedbackSchema.index({ createdAt: -1 });
26
27 module.exports = mongoose.model('Feedback', feedbackSchema);

```

6. BACKEND IMPLEMENTATION

The backend is a RESTful HTTP API implemented in Node.js using the Express.js framework (version 5). It acts as the intermediary between the React Native mobile application and the MongoDB database, enforcing data validation, routing requests, and returning structured JSON responses.

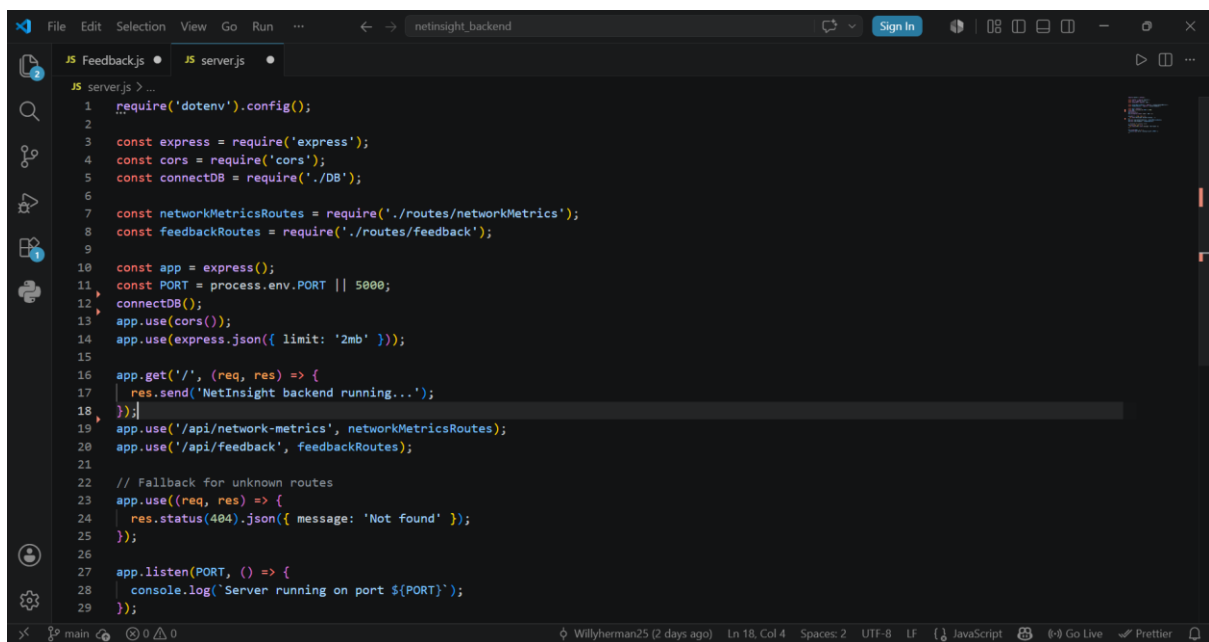
6.1. Technology Stack

Component	Technology	Version
Runtime	Node.js	LTS (v22)
Web Framework	Express.js	5.1.0
ODM Library	Mongoose	8.15.1
CORS Handling	cors	2.8.5
Env Variables	dotenv	16.4.5
Dev Reload	nodemon	3.1.0
Database	MongoDB Atlas	M0 Free Tier
Hosting	Railway	Production Environment

6.2. Entry Point – server.js

The server.js file bootstraps the Express application, connects to the database, registers

middleware, mounts route handlers, and starts the HTTP listener:



```
1 require('dotenv').config();
2
3 const express = require('express');
4 const cors = require('cors');
5 const connectDB = require('./DB');
6
7 const networkMetricsRoutes = require('./routes/networkMetrics');
8 const feedbackRoutes = require('./routes/feedback');
9
10 const app = express();
11 const PORT = process.env.PORT || 5000;
12 connectDB();
13 app.use(cors());
14 app.use(express.json({ limit: '2mb' }));
15
16 app.get('/', (req, res) => {
17   res.send('NetInsight backend running...');
18 });
19 app.use('/api/network-metrics', networkMetricsRoutes);
20 app.use('/api/feedback', feedbackRoutes);
21
22 // Fallback for unknown routes
23 app.use((req, res) => {
24   res.status(404).json({ message: 'Not found' });
25 });
26
27 app.listen(PORT, () => {
28   console.log(`Server running on port ${PORT}`);
29 });
```

6.3. API Endpoints – Network Metrics

The network metrics router (routes/networkMetrics.js) exposes three endpoints:

Method	Path	Status	Description
POST	/api/network-metrics	201 Created	Saves a single network measurement sample to the database
GET	/api/network-metrics	200 OK	Returns recent samples, optionally filtered by ?deviceId and limited by ?limit (max 500)
GET	/api/network-metrics/stats	200 OK	Returns today's aggregate averages (avgDownload, avgUpload, avgLatency, sampleCount) computed via MongoDB aggregation pipeline

The /stats endpoint uses a MongoDB aggregation pipeline to compute daily averages efficiently:

```
// GET /api/network-metrics/stats
```

```
59     const startOfDay = new Date();
60     startOfDay.setHours(0, 0, 0, 0);
61
62     const match = { timestamp: { $gte: startOfDay } };
63     if (deviceId) match.deviceId = deviceId;
64
65     const result = await NetworkMetric.aggregate([
66       { $match: match },
67       {
68         $group: {
69           _id: null,
70           avgDownloadSpeed: { $avg: '$downloadSpeed' },
71           avgUploadSpeed: { $avg: '$uploadSpeed' },
72           avgLatency: { $avg: '$latency' },
73           sampleCount: { $sum: 1 },
74         },
75       },
76     ]);
77
```

6.4. API Endpoints – Feedback

The feedback router (routes/feedback.js) exposes three endpoints with input validation:

Method	Path	Status	Description
POST	/api/feedback	201 Created	Validates and saves a user feedback entry. Returns 400 if experience, areaOfFeedback, or rating is invalid.
GET	/api/feedback	200 OK	Returns feedback entries most-recent-first. Supports optional ?resolved=true/false filter and ?limit parameter.
PATCH	/api/feedback/:id	200 OK	Updates the resolved field of a specific feedback entry, allowing administrators to mark feedback as actioned.

6.5. Input Validation

The backend performs server-side validation before persisting any data. This is a critical layer

of defence ensuring data integrity independent of the mobile client:

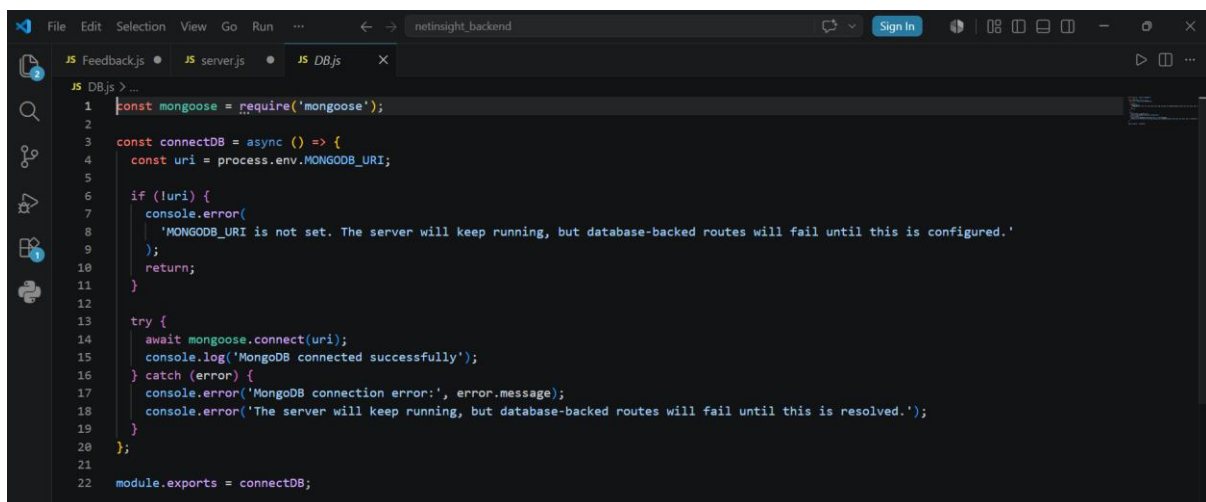
- Network Metrics: deviceId is required (400 Bad Request if absent). All other fields fall back to schema defaults if omitted.
- Feedback: experience must be one of the five valid enum values. areaOfFeedback must be non-empty. rating must be a number between 1 and 5 inclusive. Failures return descriptive 400 responses.
- Both routes catch all errors internally and return 500 Internal Server Error responses with a message field rather than exposing raw stack traces to the client.

7. CONNECTING BACKEND WITH DATABASE

The connection between the Express backend and the MongoDB Atlas cluster is managed by Mongoose and configured entirely through environment variables, following the Twelve-Factor App methodology for separating configuration from code.

7.1. DB.js – Connection Module

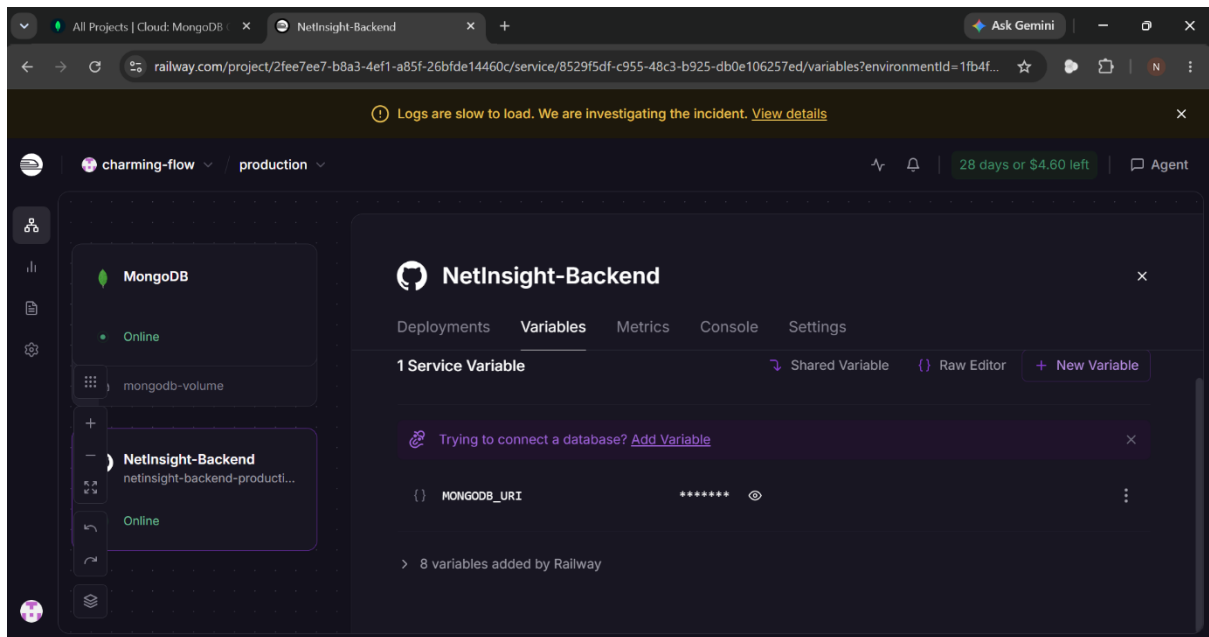
The database connection logic is encapsulated in a dedicated module (DB.js) that is imported and called once at application startup:



```
1 const mongoose = require('mongoose');
2
3 const connectDB = async () => {
4   const uri = process.env.MONGODB_URI;
5
6   if (!uri) {
7     console.error(
8       'MONGODB_URI is not set. The server will keep running, but database-backed routes will fail until this is configured.'
9     );
10    return;
11  }
12
13  try {
14    await mongoose.connect(uri);
15    console.log('MongoDB connected successfully');
16  } catch (error) {
17    console.error('MongoDB connection error:', error.message);
18    console.error('The server will keep running, but database-backed routes will fail until this is resolved.');
```

7.2. Environment Variable Configuration

The MongoDB Atlas connection string is stored as an environment variable (MONGODB_URI) and never committed to the Git repository. The connection string follows the mongodb+srv:// scheme used by Atlas:

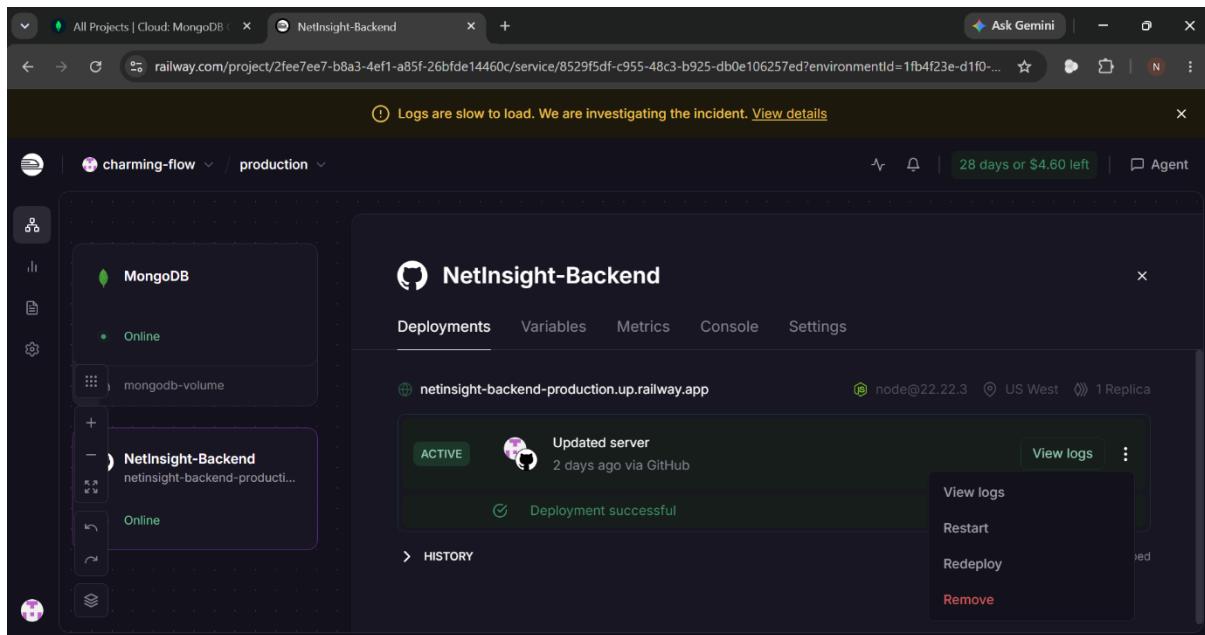


.env.example (committed to repo as a template, never the actual .env)

```
MONGODB_URI=mongodb+srv://<username>:<password>@cluster0.xxxxx.mongodb.net/NetInsight?retryWrites=true&w=majority  
PORT=5000
```

In production (Railway), the MONGODB_URI value is injected into the container as a platform-level environment variable via Railway's Variables dashboard. This approach:

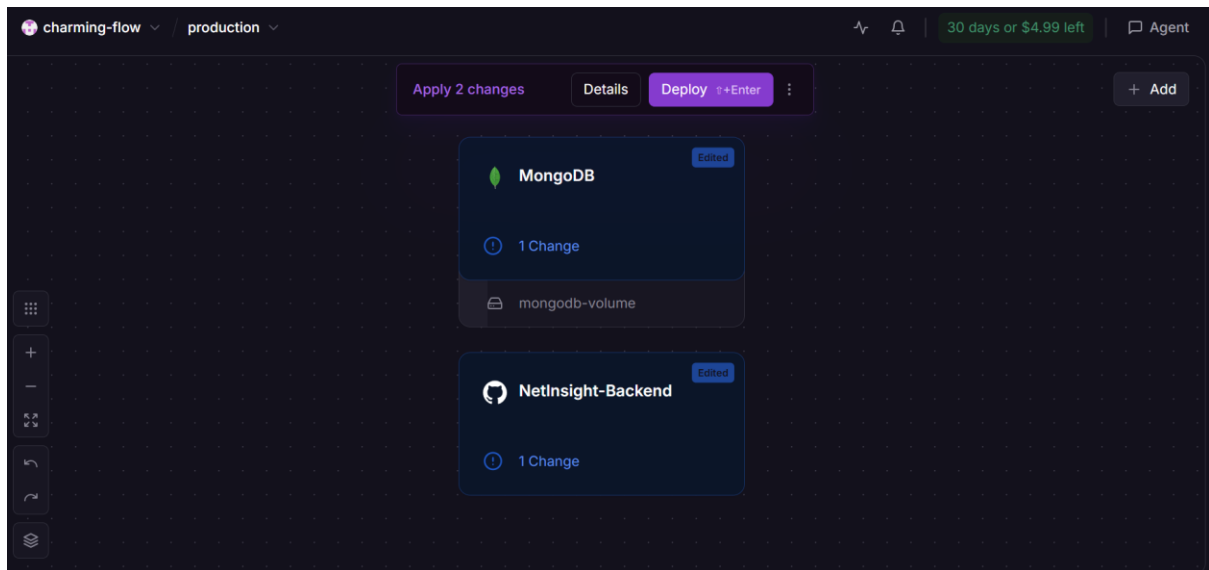
- Prevents credentials from being exposed in version control
- Allows the same codebase to connect to different databases in different environments (development, staging, production)
- Follows industry-standard security practices for credential management



7.3. Connection Lifecycle

The following sequence describes the full lifecycle from application start to a successful database-backed API call:

- Railway starts the container and runs `npm start (node server.js)`
- `dotenv.config()` is called but finds no `.env` file in production; Railway's injected environment variables are already available via `process.env`
- `connectDB()` is called, reads `process.env.MONGODB_URI`, and calls `mongoose.connect(uri)`
- Mongoose opens a persistent TCP connection pool to the MongoDB Atlas cluster using the `mongodb+srv://` DNS seed list protocol
- MongoDB Atlas authenticates the connection using SCRAM-SHA-256 with the provided credentials
- The server logs "MongoDB connected successfully" and begins accepting HTTP requests
- Incoming `POST /api/network-metrics` requests are validated, then passed to `NetworkMetric.create()` which writes to the `networkmetrics` collection
- Incoming `GET /api/network-metrics/stats` requests trigger a MongoDB aggregation pipeline that is executed in the Atlas cluster and the result is returned as JSON



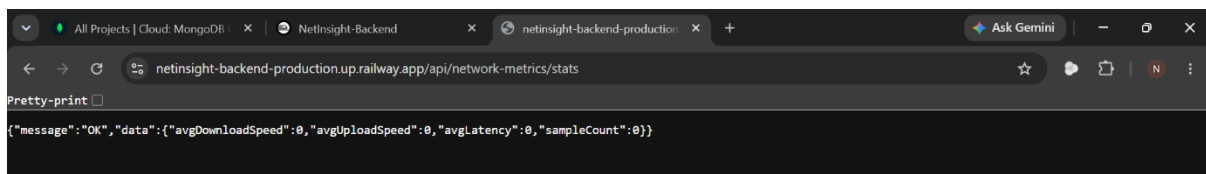
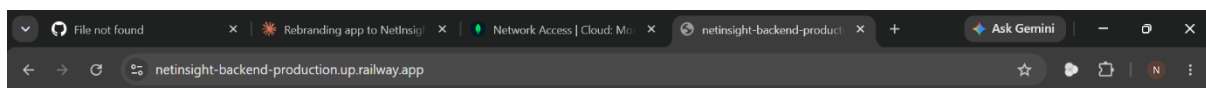
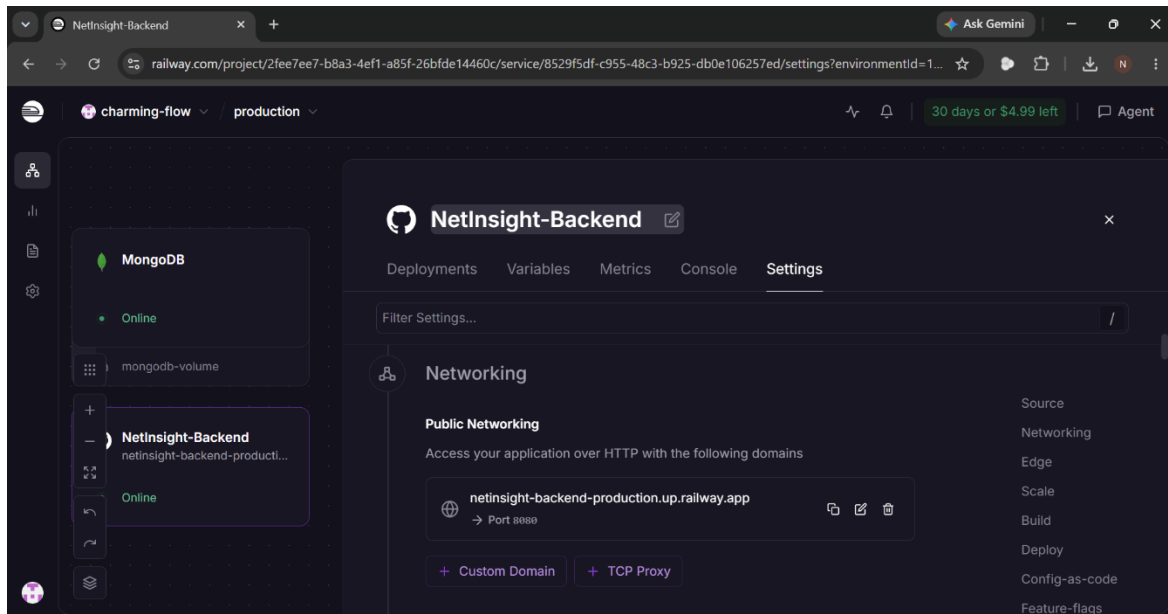
Security Considerations

The following security measures are implemented in the database-to-backend connection:

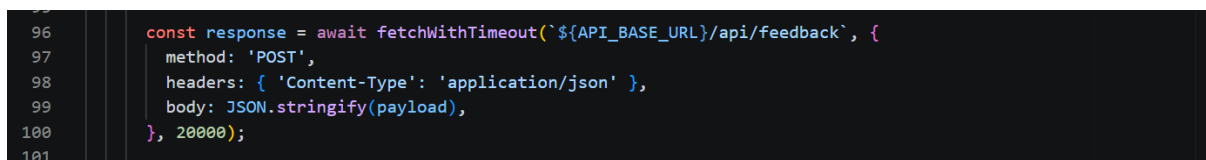
- Credentials in environment variables only: The `MONGODB_URI` (containing database username and password) is never committed to the GitHub repository. It is stored exclusively in Railway's encrypted environment variable store.
- TLS/SSL in transit: All traffic between Railway and MongoDB Atlas travels over TLS-encrypted connections enforced by the `mongodb+srv://` protocol.
- IP whitelisting: MongoDB Atlas Network Access is configured to allow connections from any IP (`0.0.0.0/0`), which is necessary because Railway does not provide static egress IPs on the free tier. In a production deployment with fixed IPs, this would be restricted.
- No `process.exit()` on DB failure: If the MongoDB connection fails, the server logs the error but continues running. This prevents a transient network hiccup from taking the entire API offline.
- CORS middleware: The `cors` package is applied globally to allow the React Native client (running on any device IP) to make cross-origin requests to the Railway-hosted API.

8. CONNECTING BACKEND TO FRONT END

After Railway web hosting had generated the URL for our backend and we tested it as seen below;



We went ahead to connect the Backend API to our Frontend Screens for them to be able to Communicate. We added a backend API on the Metrics and Feedback to store our data to the database and also added the API to be able to get the network metrics as seen below;



```

442     const response = await fetchWithTimeout(`${API_BASE_URL}/api/network-metrics`, {
443       method: 'POST',
444       headers: { 'Content-Type': 'application/json' },
445       body: JSON.stringify(payload),
446     }, 15000);
447
448     if (!response.ok) {
449       console.error('Error submitting metrics');
450     } else {
451       console.log('✅ Network metrics submitted');
452     }
453   } catch (err: any) {
454     if (err?.name === 'AbortError') return;
455     console.error('❌ Failed to submit network metrics:', err);
456   }
457 };

```

```

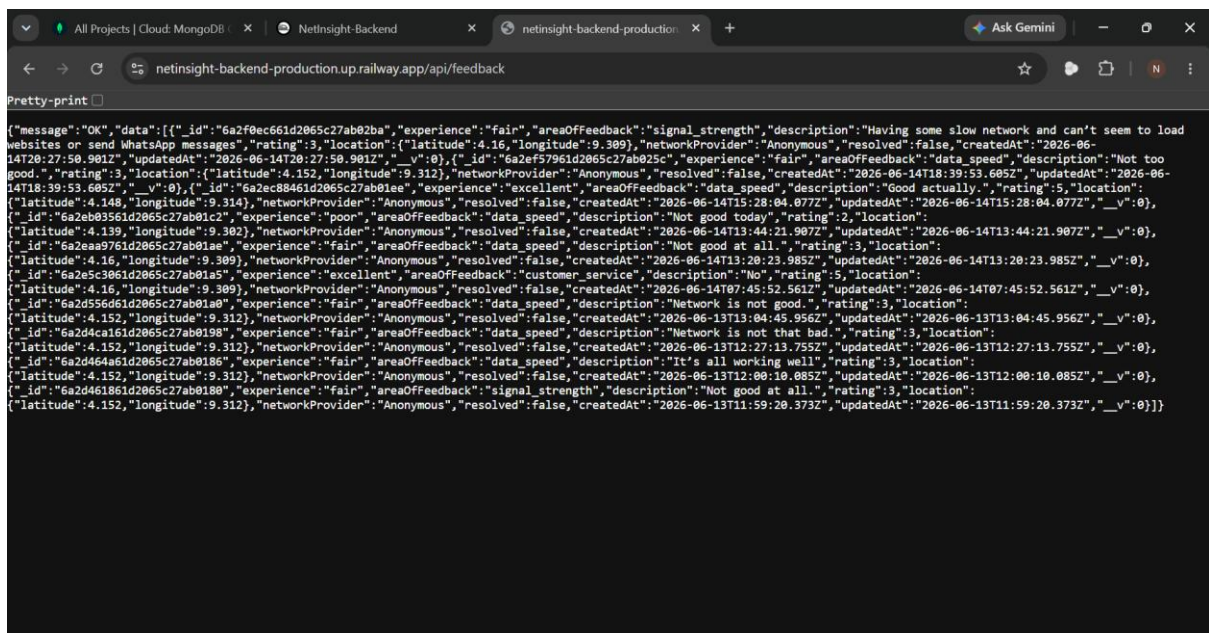
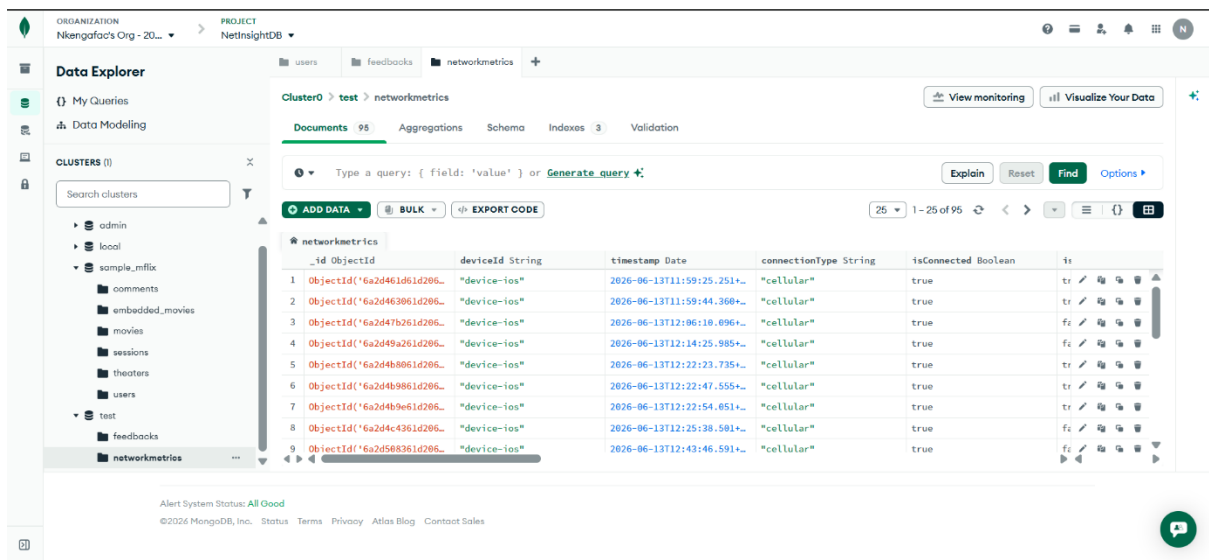
459 export const getNetworkStatsFromServer = async () => {
460   try {
461     const response = await fetchWithTimeout(`${API_BASE_URL}/api/network-metrics/stats`, {
462       method: 'GET',
463       headers: { 'Content-Type': 'application/json' }
464     }, 12000);
465
466     if (!response.ok) throw new Error('Failed to fetch stats');
467     return await response.json();
468   } catch (error) {
469     console.error("Error in getNetworkStatsFromServer:", error);
470     throw error;
471   }
472 };
473

```

After Connecting them, we concluded that our backend was successfully connected to our front be checking our database for stored data as seen below;

The screenshot shows the MongoDB Data Explorer interface. On the left, the 'Data Explorer' sidebar shows a tree view of clusters, including 'test' and 'networkmetrics'. The main panel displays the 'test' cluster with a 'feedbacks' collection. A table of documents is shown, with columns for _id, experience, areaOfFeedback, description, rating, and loca. The table contains 8 rows of data.

	_id	experience	areaOfFeedback	description	rating	loca
1	ObjectId('6a2d461861d206...')	"fair"	"signal_strength"	"Not good at all."	3	() 2 / ✎ 🔄 🗑
2	ObjectId('6a2d464a61d206...')	"fair"	"data_speed"	"It's all working well"	3	() 2 / ✎ 🔄 🗑
3	ObjectId('6a2d4ca161d206...')	"fair"	"data_speed"	"Network is not that bad."	3	() 2 / ✎ 🔄 🗑
4	ObjectId('6a2d556d61d206...')	"fair"	"data_speed"	"Network is not good."	3	() 2 / ✎ 🔄 🗑
5	ObjectId('6a2e5c3061d206...')	"excellent"	"customer_service"	"No"	5	() 2 / ✎ 🔄 🗑
6	ObjectId('6a2eaa9761d206...')	"fair"	"data_speed"	"Not good at all."	3	() 2 / ✎ 🔄 🗑
7	ObjectId('6a2eb03561d206...')	"poor"	"data_speed"	"Not good today"	2	() 2 / ✎ 🔄 🗑
8	ObjectId('6a2ec88461d206...')	"excellent"	"data_speed"	"Good actually."	5	() 2 / ✎ 🔄 🗑



9. CONCLUSION

The NetInsight database design and backend implementation provide a robust, scalable, and secure foundation for collecting and analysing mobile network quality data across Buea, Cameroon. The key design decisions — using MongoDB's document model for schema flexibility, embedding location sub-documents to avoid joins, defining performance indexes on timestamp and deviceId, and decoupling credentials from code via environment variables — collectively ensure the system is maintainable, extensible, and production-ready.

The Express.js RESTful API cleanly separates routing, data modelling, and server

configuration into distinct layers, following established Node.js conventions. The Mongoose ODM provides schema-level validation and a consistent interface to MongoDB, while the deployment on Railway with MongoDB Atlas delivers a fully managed, cloud-hosted backend that requires no server administration.

10. REFERENCES

- MongoDBDocumentation: <https://docs.mongodb.com/>
- MongooseDocumentation: <https://mongoosejs.com/docs/>
- Express.js Guide: <https://expressjs.com/>
- "Designing Data-Intensive Applications" by Martin Kleppmann.
- "MongoDB:TheDefinitive Guide" by Kristina Chodorow.
- OWASPTop10WebApplicationSecurity Risks.